UNIVERSITY
OF LONDON

Module Name: Algorithms and Data Structures 2

Module Code: CM2035

Name: Devon Maya Xavior

SID: 200618744

Date: 04 July 2024

SECTION 1 – INTRODUCTION

In this project, the Postfix++ interpreter is being developed to evaluate arithmetic expressions expressed in postfix notation, leveraging a **Stack data structure** approach. This interpreter supports variable assignments, facilitating the integration of variables within expressions. Variables are managed and stored using a **Hash Table-**based symbol table. Implemented in **JavaScript**, the interpreter handles fundamental arithmetic operations (+, -, *, /) while ensuring seamless variable management through the symbol table.

SECTION 2 - ALGORITHMS

The original algorithm to interpreting a postfix expression involves using a stack data structure to process operands (i.e., numbers or variables with values) and operators (i.e., +, -, *, /). The following is an explanation to how the algorithms for the postfix++ interpreter works. I will be using the analogy of stacking a tower of plates to explain in a non-technical manner.

STACK DATA STRUCTURE

The algorithm begins with the definition of a Stack Class, serving as the blueprint for constructing a stack. A stack is a data structure that operates on the **Last In, First Out (LIFO) principle**, managing a collection of elements where the most recently added element is the first to be removed. So in the analogy of a tower of plates, the last plate that was stacked, which is on the top of the tower, will be the first plate to be removed.

Within the Stack class, there are several actions or what we call “functions”, that we can do:

- Constructor Function:
    - When a new stack is created, it starts an empty list called ‘items’ to store the elements. In the plates analogy, the place where the new stack of plates will be placed is designated and labelled ‘items’.
- push(element) Function:
    - This function adds a new element to the top of the stack. So, with the plates analogy, a new plate will be added to the top of the stack of plates.
- pop() Function:

    - this function removes the top element from the stack; if the stack is empty, it returns a message that says “Underflow”. With the plates analogy, this means the top plate on the stack will be removed; but if there is no plate at all where the stack should be, we will be told there’s an underflow of plates, which means you’re trying to use more of something than you actually have.
- peek() Function:
    - this function returns the top element of the stack without removing it. In the plates analogy, this means that if you want to know which plate is at the top of the stack, you can have a look without removing it from the stack.
- isEmpty() Function:
    - this checks if the stack is empty, and returns the word ‘true’ if the stack does not have any elements in it; otherwise, it will return the word ‘false’. In the plates analogy, this means we are checking to see if there are no plates in the stack.
- printStack() Function:
    - this returns all the elements in the stack, in a string representation and separated by spaces. In the plates analogy, this will show us all the plates in that tower, one by one, from the top to the bottom, with each plate separated by spaces. This allows us to see all the plates in the stack, listed in order, without changing anything.

With all these functions explained, an example algorithm for a stack data structure looks like this:

Step 1: Create a new stack with no items yet, and label it ‘items’

Step 2: Since the stack is empty, let’s add an item to the stack using the push() function, and repeat this until we’re satisfied.

Step 3: Let’s see how which item is at the top of the stack using the peek() function.

Step 4: If we don’t want the top item there, we can remove it using the pop() function, and repeat the action until an error occurs which means there’s probably nothing left.

Step 5: to confirm if the stack is really empty, we can check using the isEmpty() function, if the result is ‘true’, the stack is empty, if the result is ‘false’, that means there’re still some items left in the stack.

Step 6: if it returns 'false', let's see what's left in the stack by using the printStack() function to get the list of items left.

This algorithm will be integrated into the final Postfix++ interpreter algorithm to design the source code.

HASH TABLE (SYMBOL TABLE) DATA STRUCTURE

The Hash table data structure is what we will use as the basis for the symbol table. It will store key-value pairs, where the hash function will convert keys into indices in an array (i.e., 'this.table'). This way, we can insert and retrieve values based on their keys in the table.

Below are the several functions and methods used in the algorithm of a symbol table based on the hash table structure:

- Constructor('constructor(size=26)'):
    - This creates an empty hash table with a designated size (for the purpose of this assignment, the default will be 26, following the number of letters in the alphabet i.e., A to Z)
- Hash Function ('_hash(key)'):
    - Changes a 'key' (or a variable name) into an index within the hash table (symbol table). This index shows us where the key-value pair is stored.
- Insert Method ('insert(key, value)'):
    - This adds or updates a particular key-value pair in the table.
    - It will manage collisions i.e., when more than 1 key is mapped to the same index, by the method of chaining, where arrays are used to store multiple key-value pairs at the same index (in this case, letter)
    - The Hash function is used to find the right index.
- Search Method ('search(key)'):
    - Finds the value that's matched to a given key from the hash table
    - Use the hash function to find the correct index and looks through the chain array at that index to find the right key-value pair.

Algorithm in steps:

1. First, we set up a blank space using the constructor function with several empty containers or buckets where the key-value pairs will be stored with a size limit of 26 keys.
2. When given a key, we will use the Hash function to change it into a number that determines where in the table the paired value will be stored.

3. Then we will use the insert function to put it in the table:
    a. If the key is already in the table, the key-value pairing will be updated.
    b. If the key's not found in the table, a new array will be created under that key.
4. To find out the connected value under a certain key, we use the search function. If no value is found under the specific key, null will be returned to us we will see the word 'undefined'.

The Hash function table algorithm will be integrated into the Postfix++ interpreter algorithm to create the source code.

POSTFIX++ EVALUATION ALGORITHM

The Postfix++ interpreter employs algorithms derived from both stack and hash table data structures to efficiently evaluate any given Postfix++ expression and produce accurate results. The following is how the algorithm works:

1. First, we start with an empty stack to hold the operands and intermediate results.
2. Then, we will go through the expression of tokens where each token is either the operand (number) or operator (+, -, *, /), in the postfix expression from left to right.

- If the token is an **operand (number)**, we will push it onto the stack.
- If the token is an **operator (i.e., +, -, *, /),** we will:
    - remove the top two operands from the stack
    - Apply the operator to the operands in the order they were removed
    - Then, we will push the result back onto the stack.
    - If the token is a variable, push its value onto the stack (look up the value in the symbol table).
- We will continue the process until all tokens in the postfix expression have been processed.
- After that, the stack will only contain one element, which is the result of the postfix expression after evaluation.

SECTION 3 - PSEUDOCODE

STACK DATA STRUCTURE PSEUDOCODE

```
CLASS Stack:
    FUNCTION Constructor:
        // Initialize an empty stack
        items = NEW LIST

    FUNCTION Push(element):
        // Add element to the end of the stack
        APPEND element TO items

    FUNCTION Pop:
        // Remove and return the last element of the stack
        IF LENGTH OF items EQUALS 0:
            RETURN "Underflow"
        ELSE:
            RETURN items REMOVE LAST ELEMENT

    FUNCTION Peek:
        // Return the last element of the stack
        RETURN LAST ELEMENT OF items

    FUNCTION IsEmpty:
        // Check if the stack is empty
        IF LENGTH OF items EQUALS 0:
            RETURN "true"
        ELSE:
            RETURN "false"

    FUNCTION PrintStack:
        // Return a string representation of the stack
        RETURN items JOINED BY SPACE
```

HASH TABLE DATA STRUCTURE PSEUDOCODE

```
CLASS HashTable:
    FUNCTION Constructor(size = 26):
        // Initialize a hash table with empty buckets
        this.table = NEW ARRAY OF size

    FUNCTION _hash(key):
        // Compute an index from key using a simple hash function
        RETURN key.charCodeAt(0) MOD this.table.length

    FUNCTION Insert(key, value):
        // Calculate index using hash function
        index = _hash(key)
        // Initialize bucket if it doesn't exist
        IF this.table[index] == null:
            this.table[index] = NEW LIST
            this.table[index].append([key, value])
            RETURN index
        ELSE:
            // Handle collision: linear probing
            WHILE this.table[index] NOT_EQUAL_TO undefined DO:
                index = (index + 1) MOD this.table.length
                IF index EQUALS _hash(key):
                    THROW_ERROR("Hash table overflow")
            // Insert key-value pair into the first available slot
            this.table[index] = NEW LIST
            this.table[index].append([key, value])
            RETURN index

    FUNCTION Search(key):
        // Calculate index using hash function
        index = _hash(key)
        // Search for key in bucket if it exists
        IF this.table[index] EXISTS THEN
            FOR i = 0 TO this.table[index].length - 1 DO:
                IF this.table[index][i][0] EQUALS key THEN
                    // Return value if key found
                    RETURN this.table[index][i][1]
        // Return undefined if key not found
        RETURN undefined
```

POSTFIX++ EVALUATION PSEUDOCODE

```
FUNCTION EvaluatePostfix(expression, symbolTable):
    // Initialize stack to store operands and intermediate results
    stack = new Stack()

    // Split the postfix expression into tokens
    tokens = SPLIT(expression, ' ')

    // Process each token in the expression
    FOR EACH token IN tokens:
        IF IS_NUMBER(token) THEN
            // If token is a number, push it onto the stack
            stack.push(CONVERT_TO_FLOAT(token))
        ELSE IF token IS IN ['+', '-', '*', '/'] THEN
            // If token is an operator, pop two operands from the stack
            operand2 = stack.pop()
            operand1 = stack.pop()
            result = 0

            // Perform the operation based on the operator
            SWITCH (token):
                CASE '+':
                    result = operand1 + operand2
                    BREAK
                CASE '-':
                    result = operand1 - operand2
                    BREAK
                CASE '*':
                    result = operand1 * operand2
                    BREAK
                CASE '/':
                    result = operand1 / operand2
                    BREAK

            // Push the result back onto the stack
```

```
            stack.push(result)
        ELSE:
            // If token is a variable, look up its value in the symbol table
            value = symbolTable.search(token)
            IF value IS NOT UNDEFINED THEN
                stack.push(value)
            ELSE:
                // Throw an error if the variable is undefined
                THROW_ERROR("Undefined variable: " + token)

// Return the final result from the stack
RETURN stack.pop()
```

SECTION 4 – DATA STRUCTURES

STACK

The stack data structure is chosen for the evaluation portion of the Postfix interpreter because it is a natural fit for the Postfix notation. The stack data structure naturally aligns with the Postfix notation format by allowing operands to be added on and removed in a way that follows the order of operations without needing additional parentheses.

It is also a space efficient structure to use as it only needs space proportionate to the number of operands and operators in the expression – O((n), where n is the number of tokens in the postfix expression. Intermediate results are used again, reducing the amount of memory required. The maximum size of the stack occurs when the largest number of operands minus operators is encountered.

HASH TABLE (SYMBOL TABLE)

When considering what data structure to use for the symbol table, I was pondering between Hash Table and Expression Tree. The factor that won me over to Hash table was the time complexity. The average time complexity for the Hash Table is O(1) for the INSERT and SEARCH operations, whereas it is O(n) for Expression Tree, hence, the latter needs more time to traverse through the input and the time grows linearly with the size of the input. Also, using a Hash table means we can directly access the value of a variable without traversing a tree. Therefore, the Hash table was chosen in the end.

SECTION 5 - IMPLEMENTATION

LINK TO VIDEO SHOWING EXECUTION OF SOURCE CODE

COMMENTED SOURCE CODE BELOW

```
class Stack {

    constructor() {

        // Initialize an empty array to store stack items

        this.items = [];

    }


    push(element) {

        // Push an element to the stack

        this.items.push(element);

    }


    pop() {

        // Pop an element from the stack if not empty, otherwise return "Underflow"

        if (this.items.length === 0) return "Underflow";

        return this.items.pop();

    }


    peek() {

        // Peek at the top element of the stack without removing it

        return this.items[this.items.length - 1];

    }
```

```
    isEmpty() {
        // Check if the stack is empty
        return this.items.length === 0;
    }

    printStack() {
        // Print all elements in the stack
        return this.items.join(' ');
    }
}

class HashTable {
    constructor(size = 26) {
        // Initialize the hash table with the given size, default is 26
        this.table = new Array(size);
    }

    _hash(key) {
        // Compute the hash index for a given key
        return key.charCodeAt(0) % this.table.length;
    }

    insert(key, value) {
        // Insert a key-value pair into the hash table
        const index = this._hash(key);
        if (!this.table[index]) {
            this.table[index] = [];
        }
        for (let i = 0; i < this.table[index].length; i++) {
```

```
            if (this.table[index][i][0] === key) {
                this.table[index][i][1] = value;
                return;
            }
        }
        this.table[index].push([key, value]);
    }

    search(key) {
        // Search for a value by its key in the hash table
        const index = this._hash(key);
        if (this.table[index]) {
            for (let i = 0; i < this.table[index].length; i++) {
                if (this.table[index][i][0] === key) {
                    return this.table[index][i][1];
                }
            }
        }
        return undefined;
    }
}

function main() {
    // Create a new symbol table
    let symbolTable = new HashTable();

    // Assign values to variables
    assignVariable('A 3', symbolTable);
    assignVariable('B 4', symbolTable);
```

```
    // Evaluate postfix expressions
    console.log(evaluatePostfix('A B +', symbolTable)); // Output: 7
    console.log(evaluatePostfix('A B *', symbolTable)); // Output: 12
}

main();

function evaluatePostfix(expression, symbolTable) {
    // Create a new stack for evaluation
    let stack = new Stack();
    // Split the postfix expression into tokens
    let tokens = expression.split(' ');



    // Iterate through each token
    tokens.forEach(token => {
        if (!isNaN(token)) {
            // If the token is a number, push it to the stack
            stack.push(parseFloat(token));
        } else if (['+', '-', '*', '/'].includes(token)) {
            // If the token is an operator, pop two operands from the stack
            let operand2 = stack.pop();
            let operand1 = stack.pop();
            let result;
            // Perform the operation and push the result back to the stack
            switch (token) {
                case '+':
                    result = operand1 + operand2;
```

```
                    break;
                case '-':
                    result = operand1 - operand2;
                    break;
                case '*':
                    result = operand1 * operand2;
                    break;
                case '/':
                    result = operand1 / operand2;
                    break;
            }
            stack.push(result);
        } else {
            // If the token is a variable, look it up in the symbol table
            let value = symbolTable.search(token);
            if (value !== undefined) {
                // Push the variable's value to the stack
                stack.push(value);
            } else {
                // Throw an error if the variable is undefined
                throw new Error(`Undefined variable: ${token}`);
            }
        }
    });

    // Return the final result from the stack
    return stack.pop();
}
```

```
function assignVariable(expression, symbolTable) {
    // Split the assignment expression into variable and value
    let tokens = expression.split(' ');
    let variable = tokens[0];
    let value = parseFloat(tokens[1]);
    // Insert the variable and value into the symbol table
    symbolTable.insert(variable, value);
}
```

SECTION 6 – DEFECTS AND REMEDIES

EXTENDED OPERATOR AND FUNCTIONALITY SUPPORT

One major limitation of the current implementation is that it only supports basic arithmetic operators. To improve functionality, I will probably extend it to include additional operators and mathematical functions like modulus, exponential and trigonometric functions. This update will make the tool more robust and versatile for users needing complex mathematical operations.

DYNAMIC VARIABLE NAMESPACE

The current implementation limits variable names to letters A to Z, which can be restrictive when dealing with many variables. To improve flexibility, I will probably improve it to contain dynamic variable names. This means checking if variables exist in the symbol table as needed, which would better manage a larger range of variables if there is enough memory space for it.

SPEEDING UP ACCESS TO VARIABLE

Another issue is that the system doesn't optimize repeated variable access. Adding a cache for frequently used variables could speed things up. This change would cut down on the time spent looking up variables in the symbol table over and over again. Overall, it would make the interpreter work faster, especially with big data or complicated expressions.

CONCLUSION

It was not easy when it comes to implementation of the interpreter, especially with nested expressions and handling large postfix expressions efficiently. I might use the Expression Tree data structure for the evaluation portion of the source code in the future to handle operator order better and also to handle more complex variables.